



Food-Image Classification



Project Overview ;

GROUP MEMBERS

- Savins Nanyaemuny
- Linet Lydia
- Imran Mahfoudh
- Antony Ekeno
- Noel Christopher

This project is a food image classification model built using TensorFlow and trained on the Food 101 dataset. The goal is to develop a deep learning model capable of accurately classifying images into one of 101 different food categories. This project is particularly useful for applications in restaurant automation, diet tracking apps, and food delivery services that require image-based food recognition.



Business Understanding

Safety First: Ensuring accurate food classification reduces errors in menu categorization, automated checkouts, and dietary recommendations, promoting safety in food handling and consumption.

Seamless Automation: An AI-powered model with over 75% accuracy will streamline restaurant operations, enhance food delivery workflows, and improve user experiences in nutrition apps, minimizing human error and boosting efficiency.

Scalability in Real-World Scenarios: The model will be designed to handle diverse food categories, ensuring robust performance across varied environments—from bustling restaurant kitchens to large-scale food delivery platforms."



Data Understanding

The [Food-101](#) dataset, utilized for the Food classification project, is a large-scale image dataset curated for food classification tasks using TensorFlow. It contains 101,000 images, with 1,000 images for each of the 101 food categories, ranging from dishes like pizza and sushi to apple pie and ramen.

The dataset is structured into two main subsets:

- **Training Set:** 75,750 images, with approximately 750 images per class, including some noisy labels and slight misclassifications to simulate real-world conditions.
- **Testing Set:** 25,250 images, offering 250 images per class, manually cleaned to ensure accurate evaluation of model performance.

The images are RGB with varying dimensions, typically around 512x512 pixels, and include a mix of backgrounds and lighting conditions.





Inspecting the Data

To build a robust **food classification model**, it's crucial to thoroughly understand the **Food-101** dataset. This involves:

- **Class Names:** The dataset includes **101 distinct food classes**, from **apple pie** to **sushi**, ensuring our model learns to differentiate between a wide variety of dishes.
- **Data Type:** The input data consists of **image tensors**, typically stored as **float32** after normalization, allowing for efficient processing by the **CNN model**.
- **Label Format:** Labels are **label encoded**, meaning each class is represented by a unique integer (e.g., **0 for apple pie**, **1 for sushi**, etc.), simplifying classification tasks.
- **Label Consistency:** By cross-referencing **image labels** with **class names**, we ensure accuracy, preventing mismatches that could negatively impact the model's learning.

Data Preprocessing

To prepare the Food-101 dataset for the Food Vision 101 project, key preprocessing steps were applied to enhance model performance.

- **Data augmentation** (e.g., flipping, rotating, adjusting brightness) was used to expose the model to varied scenarios, improving its ability to recognize foods in real-world settings.
- All images were reshaped to **224x224 pixels**, ensuring uniformity and speeding up processing.
- A **batch size of 32** allowed the model to learn from multiple images simultaneously, boosting efficiency.
- Finally, **prefetching** was implemented to prepare the next batch of data while processing the current one, reducing delays and optimizing training speed.

These steps together created a robust and efficient training pipeline for high-accuracy food classification.





Modelling

For the **modeling phase** of the **Food Vision 101** project, we utilized both the **EfficientNetB0** and **EfficientNetB7** architectures, adopting a **two-step approach** to identify the **best-performing model**:

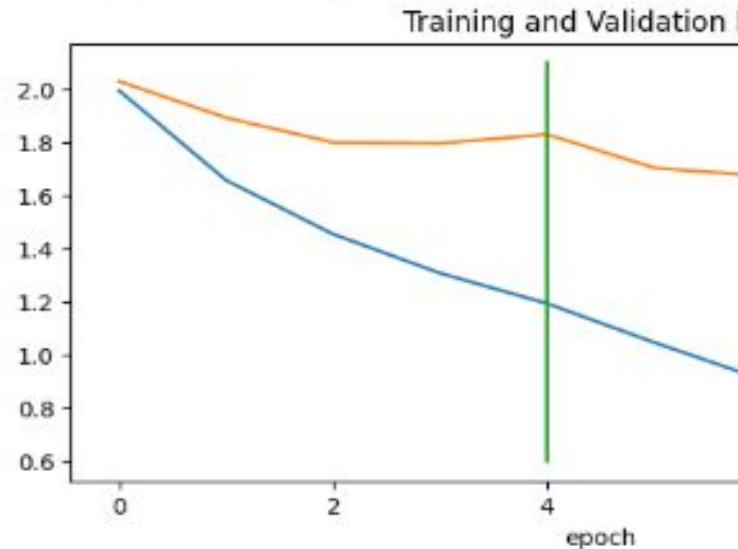
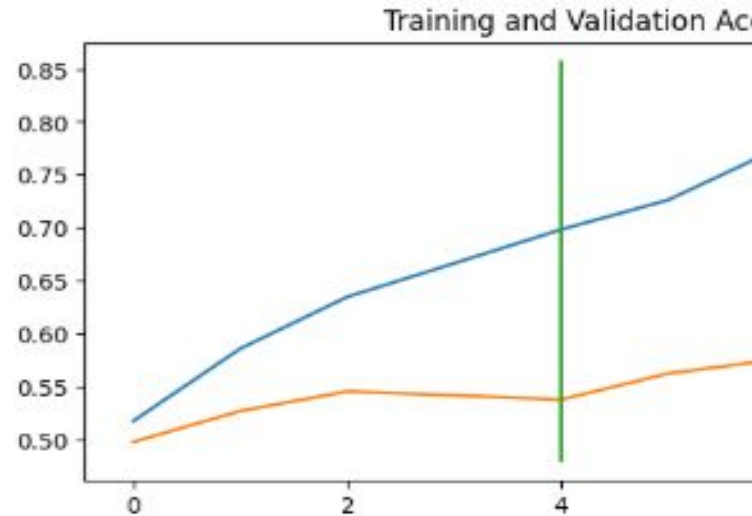
1. **Feature Extraction (Baseline Model)**: Initially, we used **pre-trained EfficientNet models** as feature extractors, keeping all layers **frozen**. This approach allowed the model to leverage **pre-learned features** from large image datasets without modifying the internal weights. The baseline model served as a **benchmark**, offering a quick and stable start to measure initial performance.
2. **Fine-Tuning**: After establishing a solid baseline, we **unfroze the top layers** of the model and **fine-tuned** them on our specific **Food-101** dataset. This step allowed the model to adapt **pre-trained features** to our domain, learning subtle distinctions between food classes. **Fine-tuning** often brings a performance boost, as the model can refine its understanding based on our unique data.



Model Evaluation

Comparing History Curves: Throughout training, we tracked the **loss** and **accuracy curves** for both **training** and **testing datasets**. By comparing these curves, we gained insights into:

1. **Model Performance:** A steadily decreasing loss and increasing accuracy indicated effective learning.
2. **Overfitting Signs:** If the training accuracy improved while test accuracy plateaued or declined, it signaled overfitting, helping us adjust the model accordingly.
3. **Model Choice:** The model with the best balance between train and test performance—achieving high accuracy with low overfitting—was chosen





Remedies of Overfitting

We implemented several strategies to enhance the model's generalization capability, including:

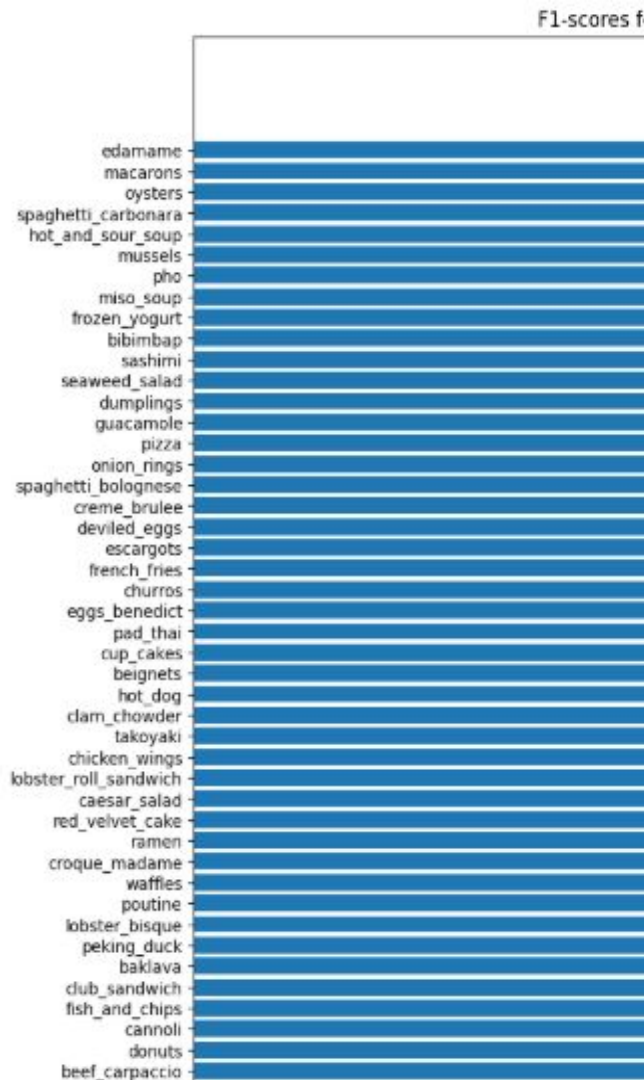
- **Data Augmentation:** We applied **techniques** like **random rotations, flips, shifts, and zooms** to **artificially expand the training dataset**, helping the model **learn robust features** rather than **memorizing specific images**.
- **Early Stopping:** We monitored the **validation loss** and **halted training** when it **stopped improving**, preventing the model from **over-training** on the **training data**.
- **Regularization Techniques:** We incorporated **Dropout layers**, which **randomly deactivate neurons** during **training**, reducing the risk of the model becoming **too reliant on specific pathways**.
- **Transfer Learning with Fine-Tuning:** Initially, we used **feature extraction** to leverage **pre-trained weights** without **unfreezing layers**. During **fine-tuning**, only the **top layers** were updated, maintaining the **general features** learned from the **base model** and minimizing **overfitting risks**.
- **Batch Normalization:** Implemented **batch normalization layers** to **stabilize training**, allowing the model to **learn faster** and **generalize better**.



Model Evaluation

The **F1-score** visualization for the **101 different food classes** provides a detailed view of the **model's performance** across each category. The **F1-score** combines **precision** and **recall**, offering a balanced measure of **model accuracy** in classifying each food type.

From the **chart**, it is evident that certain classes, such as **edamame**, **macarons**, and **oysters**, achieved **higher F1-scores**, indicating the model's **strong performance** in **accurately predicting these categories**. This could be due to **distinctive features** in the images or a **balanced representation** in the **training data**.





Recommendations

- **Enhance Business Framing:** Clearly defining project goals and intended use cases ensures that the model aligns with business needs and user expectations. This clarity improves communication with stakeholders and increases project adoption.
- **Improve Model Explainability:** Implementing SHAP/LIME, confusion matrices, and misclassification analysis helps in understanding how the model makes predictions. This transparency increases trust in the model, facilitates debugging, and allows stakeholders to make informed decisions based on the model's outputs.
- **Address Class Imbalances:** Using weighted loss functions or data augmentation mitigates the impact of underrepresented classes in the dataset. This step improves overall model performance, ensuring fair and accurate classifications across all food categories.
- **Optimize for Deployment:** Including API integration or inference pipelines makes the model ready for real-world applications. This step ensures smooth integration into food-related platforms, improving usability and increasing the model's impact.



Conclusions

- The project effectively demonstrates food classification using machine learning.
- While technically strong, adding interpretability and business context will make it more impactful.
- The next steps focuses on refining model deployment and ensuring usability in its intended application.

Challenges & Problems Faced During the Project

Data Imbalance Issues: Some food categories were underrepresented, causing the model to struggle with less common items.

- **Solution:** Data augmentation and weighted loss functions were applied to balance class distribution.

Misclassification of Visually Similar Foods: Items like different types of pasta, soups, or curries were frequently confused.

- **Solution:** Feature engineering and pre-trained models (EfficientNet, MobileNetV2) were explored to improve accuracy.

Deployment & Integration Concerns: Ensuring that the trained model could be efficiently deployed in real-world applications such as mobile apps, web platforms, or restaurant POS systems.

- **Solution:** API integration and inference pipelines were recommended for



SUCCESS STORY & ACHIEVEMENTS

Achieved ~76% Accuracy

- A significant improvement over traditional image classification models, providing reliable predictions for various food types.

Tested on Custom Real-World Images

- Successfully classified food images outside of the training dataset, demonstrating the model's potential for real-world use.

Enhanced Business Use Cases

- Automated **food categorization, checkouts, and dietary tracking**, proving its value for restaurants and food delivery platforms.

Scalability for Future Deployment

- Model architecture designed to **integrate with food industry applications**, allowing businesses to streamline operations and improve efficiency.



Thanks!

Any questions?